GOVERNMENT COLLEGE OF ENGINEERING, AMRAVATI

(An Autonomous Institute of Government of Maharashtra)

OPERATING SYSTEMS MINI PROJECT

OS Lab Studio

Interactive Operating Systems Algorithm Simulator

CPU Scheduling • Page Replacement • Disk Scheduling
Deadlock Avoidance • Semaphore Synchronization • File Allocation

PROJECT SUBMISSION DETAILS

Course Code	CS1502
Course Name	Operating System
Department	Computer Science and Engineering
Student Name	_____
Roll Number	_____
Academic Year / Semester	_____

Department of Computer Science and Engineering

A browser-based, offline-ready educational simulator

Abstract

Operating Systems coordinate CPU time, main memory, secondary storage, and shared resources. The behavior of many OS policies can be difficult to understand from definitions alone. **OS Lab Studio** is a browser-based educational application that visualizes six important OS lab areas: CPU scheduling, page replacement, disk scheduling, deadlock avoidance using the Banker's algorithm, semaphore synchronization using the bounded Producer–Consumer problem, and file allocation using Contiguous, Linked, and Indexed methods. Users can enter workload values, execute an algorithm, inspect timelines or state tables, compare policies, and export results as CSV. The project is built with HTML, CSS, and vanilla JavaScript and runs locally without a database or package installation. It is a simulator for learning and does not modify or inspect the host operating system's actual scheduling or memory management.

1. Introduction

An operating system is system software that manages computer hardware and provides services to application programs. Processes compete for CPU time; programs access virtual memory; I/O requests compete for disk-head service; and processes may request limited resources. Different algorithms optimize different objectives, so it is useful to observe their decisions on the same input.

OS Lab Studio integrates these ideas into one small application. Instead of providing only one hard-coded example, it accepts editable inputs and recalculates the result. The visualization and summary metrics make algorithm behavior easier to demonstrate during an OS laboratory practical or viva.

2. Problem Statement

Students often learn OS algorithms from formulas, static tables, or handwritten traces. Repeating a calculation after changing an arrival time, page reference, cylinder request, or resource request can be slow and error-prone. The goal is to provide a single interactive tool that performs these calculations consistently and makes its decisions visible, including semaphore transitions and blocking in a shared-buffer scenario.

3. Objectives

1. Demonstrate how CPU scheduling policies select processes from a ready queue.
2. Calculate waiting time, turnaround time, response time, throughput, and CPU utilization.
3. Visualize page hits, page faults, frame contents, and page eviction decisions.
4. Compare disk scheduling policies using ordered cylinder paths and total head movement.
5. Determine safe states and safe sequences using the Banker's safety algorithm.
6. Evaluate whether a resource request can be granted without moving the system into an unsafe state.
7. Demonstrate semaphore operations and the Producer–Consumer synchronization problem, including full/empty buffer blocking.
8. Compare Contiguous, Linked, and Indexed file allocation and visualize fragmentation and index-block overhead.
9. Provide a clear, responsive interface and allow results to be exported for observation tables.

4. Scope

The application is designed for teaching and small laboratory experiments. It covers six interactive algorithm labs and a reference page for common OS concepts and formulas. It does not implement a real kernel, perform hardware scheduling, access physical memory or disk queues, or model every detail of a modern operating system.

5. Tools and Technologies

Technology	Purpose
HTML5	Application layout, inputs, tables, navigation, and help dialog
CSS3	Responsive dashboard, dark visual theme, charts, state indicators,

Technology	Purpose
	and print styling
JavaScript	Scheduling logic, page replacement, disk request ordering, Banker's algorithm, validation, and result rendering
Browser Blob API	Downloading output tables as CSV
Browser print API	Printing or saving the concepts reference page as PDF

No build tool, backend, database, framework, or external library is required.

6. System Design

6.1 High-level architecture

The application follows a lightweight client-side structure:

1. **Presentation layer:** `index.html` provides the app shell; `styles.css` styles navigation, controls, charts, tables, and responsive layouts.
2. **Input and validation layer:** JavaScript reads form values and validates numeric ranges, resource constraints, and request strings before simulation.
3. **Algorithm layer:** JavaScript routines calculate CPU schedules, page-frame states, disk paths, safe sequences, semaphore transitions, and file block allocation.
4. **Visualization layer:** Results are rendered into Gantt segments, frame tables, a disk movement plot, resource matrices, buffer/semaphore traces, disk-block maps, summary statistics, and comparison bars.
5. **Export layer:** The most recent result can be downloaded in CSV format.

6.2 Module 1 — CPU Scheduling

The user supplies a process ID, arrival time, burst time, and priority. Supported algorithms are FCFS, non-preemptive SJF, preemptive SRTF, non-preemptive Priority, preemptive Priority, and Round Robin. Idle periods are represented in the Gantt chart if no process has arrived yet. Preemptive algorithms simulate CPU allocation in unit-time steps; Round Robin uses a ready queue and configurable positive integer time quantum.

Metrics:

- Turnaround Time (TAT) = Finish Time − Arrival Time
- Waiting Time (WT) = TAT − Burst Time
- Response Time (RT) = First CPU Start − Arrival Time
- Average metric = Sum of the corresponding process metric ÷ Number of processes
- CPU utilization = Total burst time ÷ finish time from t = 0 × 100
- Throughput = Number of completed processes ÷ finish time

6.3 Module 2 — Page Replacement

The user enters a page reference string and frame count. On each reference, the simulator checks whether the page is already resident. A hit leaves the frame assignment unchanged. A page fault loads the page into an empty frame or evicts a page according to FIFO, LRU, or Optimal policy.

- **FIFO:** evict the page that entered memory first among the currently resident pages.
- **LRU:** evict the resident page whose most recent use is farthest in the past.
- **Optimal:** evict the resident page whose next use is farthest in the future, or which is never used again.

The step table reports frame contents, hit/fault status, evicted page, and cumulative page faults. Hit rate and fault rate are calculated from all references.

6.4 Module 3 — Disk Scheduling

The user enters the initial head position, cylinder range, direction, and pending request queue. The simulator returns a service path and sums the absolute movement between consecutive head positions.

- **FCFS:** service requests in input order.
- **SSTF:** select the pending request closest to the current head.
- **SCAN:** move toward the selected disk boundary, then reverse to service requests on the other side.

- **C-SCAN:** service in one direction, travel to the boundary, wrap to the opposite boundary, then continue in the same service direction.
- **LOOK:** reverse at the final pending request in the current direction rather than traveling to the physical boundary.
- **C-LOOK:** jump between the outermost pending requests and continue in one direction.

Total movement includes boundary travel and the circular wrap in C-SCAN/C-LOOK. This convention is visible in the result table.

6.5 Module 4 — Deadlock Avoidance

The Banker's algorithm evaluates whether the system can finish all processes in some order without exceeding the resources currently available. The Need matrix is derived as:

Need = Maximum − Allocation

Safety procedure:

1. Set `Work = Available` and all Finish flags to false.
2. Find an unfinished process whose Need vector is less than or equal to Work for every resource type.
3. Assume that process completes, add its Allocation back to Work, and mark it finished.
4. Repeat until all processes finish or no additional process can be selected.
5. If every process finishes, the state is safe and the selected order is a safe sequence; otherwise the state is unsafe.

For the request test, the tool checks that Request does not exceed Need or Available, applies the request temporarily, then evaluates the resulting state. The original input matrices are not permanently changed by a request test.

6.6 Module 5 — Semaphore Synchronization (Producer–Consumer)

A bounded buffer is shared by a producer and a consumer. The lab starts with a configurable capacity of 2–6 slots and tracks three semaphore values:

- **`empty`** is a counting semaphore for available buffer slots; it starts at the buffer capacity.
- **`full`** is a counting semaphore for items available to consume; it starts at zero.
- **`mutex`** is a binary semaphore that protects the critical section; it starts at one (unlocked).

For a successful producer operation, the trace records `wait(empty)`, `wait(mutex)`, inserting an item, `signal(mutex)`, and `signal(full)`. A consumer performs the complementary sequence with `wait(full)`, `wait(mutex)`, removing an item, `signal(mutex)`, and `signal(empty)`. If the buffer is full, a producer attempt is shown as blocked; if empty, a consumer attempt is shown as blocked. The operation history, buffer slots, semaphore values, and event-level trace update together, and the trace can be exported to CSV.

This is a deterministic, step-by-step teaching model: it demonstrates semaphore ordering and the conditions under which an operation would block, but it does not run concurrent OS threads or model an actual kernel scheduler.

6.7 Module 6 — File Allocation Visualizer

The file-allocation lab models a small disk as numbered blocks, some of which may be reserved by the system. Users define files and the number of data blocks requested by each file, then select Contiguous, Linked, Indexed, or Compare all.

- **Contiguous allocation:** finds one consecutive run large enough for the whole file. The sample illustrates external fragmentation: the largest free run can be smaller than a file even when the total number of free blocks is sufficient.
- **Linked allocation:** assigns scattered free blocks and records their chain order. It does not need a continuous run, but sequential access follows the links.
- **Indexed allocation:** reserves an index block for each file and assigns its data blocks separately. The index block stores the address list and incurs extra block overhead.

The output includes a visual block map, per-file success/failure status, data block locations, index blocks, total free blocks, largest free run, and CSV export. The allocation routines do not leave partial allocations behind for a file that fails.

7. Algorithmic Complexity

Let *n* be the number of processes, *r* the number of page references, *f* the number of frames, and *q* the number of disk requests.

Component	General time cost in this implementation	Auxiliary space
FCFS / SJF / Priority CPU scheduling	$O(n^2 \log n)$ due to repeated sorting of the ready set	O(n) plus the Gantt segments
SRTF / Preemptive Priority	$O(T \cdot n \log n)$, where T is simulated elapsed CPU time	O(n + segments)
Round Robin	O(number of time slices · n) with this small-workload simulation	O(n + segments)
Page replacement	FIFO/LRU: $O(r \cdot f)$; Optimal: $O(r^2 \cdot f)$ because it scans the remaining future reference string for each candidate page	$O(f + r \cdot f)$ for recorded steps
Disk FCFS	O(q)	O(q)
Disk SSTF / SCAN family	SSTF: $O(q^2 \log q)$ due to repeated sorting; SCAN-family: $O(q \log q)$ for sorting	O(q)
Banker's safety check	$O(n^2 \cdot m)$ in the worst case, where m is resource types	$O(n \cdot m)$
Semaphore trace	$O(k \cdot c)$ for k operation attempts and buffer capacity c (bounded at 6)	$O(k \cdot c)$ to retain the event trace
File allocation	$O(f \cdot b)$ worst case, where f is files and b is disk blocks; contiguous allocation scans for free runs	$O(b + f \cdot b)$ including block map and per-file results

The workload limits are intentionally small to keep the visualization readable and responsive.

8. Validation and Test Results

The project includes manual test cases in `TEST_CASES.md`. The sample workload results used to verify the implementation include:

- CPU sample: all six schedulers finish at time 16; SRTF has average waiting time 3.00 for this workload.
- Page reference string `7 0 1 2 0 3 0 4 2 3 0 3 2` with 3 frames produces 10 FIFO faults, 9 LRU faults, and 7 Optimal faults.
- Disk queue `98 183 37 122 14 124 65 67`, initial head 53, cylinders 0–199, direction right: FCFS 640, SSTF 236, SCAN 331, C-SCAN 382, LOOK 299, and C-LOOK 322 cylinders of movement under this implementation's stated boundary/wrap convention.
- The classic five-process Banker example yields a safe sequence `P1 → P3 → P4 → P0 → P2`.
- Synchronization guided demo: 10 attempted operations, with 8 successful and 2 blocked; the final state has `empty = 3`, `full = 0`, and `mutex = 1`. The blocked cases are a producer encountering a full buffer and a consumer encountering an empty buffer.
- File-allocation sample: with 24 blocks and reserved indices 4, 9, 14, and 19, the Contiguous method allocates 4 of 5 files (largest free run 4); Linked allocates all 5 files; Indexed allocates 4 because the remaining 4 blocks cannot fit the fifth file plus its index block.
- The request `[1, 0, 2]` from P1 is safe for the sample state; the same vector from P0 is rejected because the tentative state is unsafe.

9. Advantages

- Six syllabus-relevant interactive labs are available in one application, including process synchronization/IPC and file allocation.
- Inputs are editable and outputs update without recompiling code.
- Visual timelines and state tables make individual algorithm decisions explainable.
- Comparison mode shows how policy choice changes results for the same workload.
- CSV exports support observation tables and further analysis.
- Offline-ready and free of package installation, backend, and database requirements.
- Responsive design supports laptop and mobile-width screens.

10. Limitations

- It is an educational model, not a kernel implementation or real system monitor.
- CPU policies use simplified integer time units and do not include context-switch overhead.
- Optimal page replacement uses future knowledge and is used for comparison rather than realistic online control.
- Disk-scheduling results depend on the selected cylinder range, initial head, direction, and the boundary/wrap convention.
- The Banker's resource model has three resource types and five sample processes; adding a larger number of resources would require extending its UI.

11. Future Enhancements

Possible extensions include animated step-by-step playback, adjustable Banker resource dimensions, additional synchronization problems such as Readers–Writers, directory and permission models, persistent lab runs, and downloadable per-experiment reports.

12. Conclusion

OS Lab Studio demonstrates how operating-system policies transform inputs into execution order, page-frame state, disk-head movement, or resource-allocation safety decisions. Combining interactive inputs, visual outputs, formulas, comparisons, and CSV export makes it useful for practical demonstrations and viva preparation. The project meets its learning objective by translating core OS algorithms into observable and reproducible browser simulations.

References

1. Abraham Silberschatz, Peter B. Galvin, and Greg Gagne, *Operating System Concepts*, 8th Edition, Wiley India, 2009.
2. William Stallings, *Operating Systems: Internals and Design Principles*, 6th Edition, Pearson Education, 2009.
3. Andrew S. Tanenbaum, *Modern Operating Systems*, 3rd Edition, Prentice Hall India, 2003.